№5 ООП

План

- Отношения между классами
  - Диаграммы классов
  - Виды отношений
    - Ассоциация
    - Агрегация
    - Композиция
    - Зависимость
    - Наследование

Отношения между классами

Диаграммы классов

Класс в диаграмме отражается как блок из трёх полей:

- *Name*.
- *field:type*.
- *method():type*.

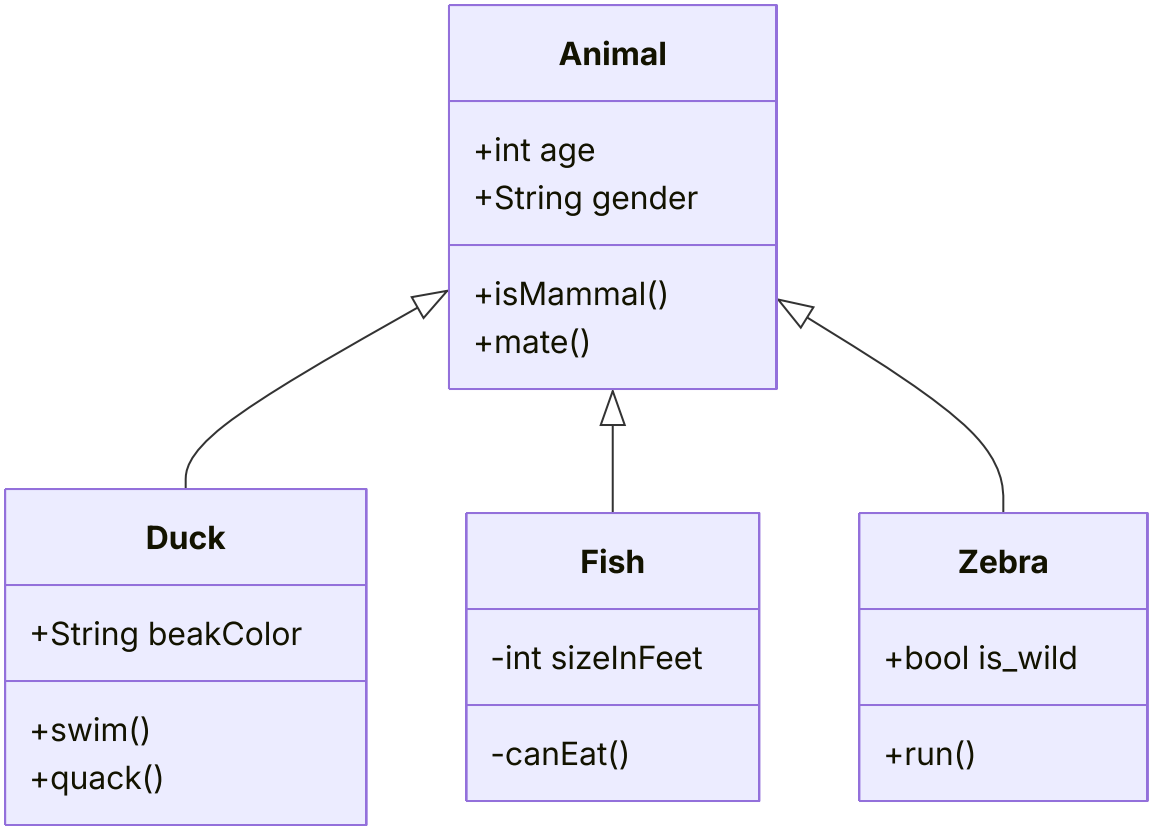
Duck
+String beakColor
+swim() +quack()

Внутри самих полей используется разделение на уровни доступа:

- `+` - *public*.
- `-` - *private*.

- `#` - *protected*.

Диаграмма классов состоит из классов и их отношений.



Есть две нотации описания классов:

- *CamelCase*.
- *Snake_case*.
  При этом принято называть *переменные с маленькой* буквы, а *типы с большой*.

Виды отношений

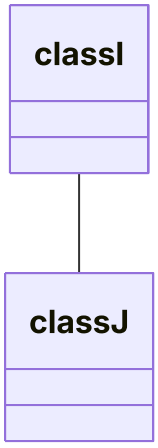
Всего пять видов *обязательных* отношений:

- *Ассоциация*.
- *Агрегация*.
- *Композиция*.
- *Зависимость(использование)*.
- *Наследование*.
  Дополнительные виды:
- *Мета-классицирование*.

- *Инстанцирование*

Ассоциация

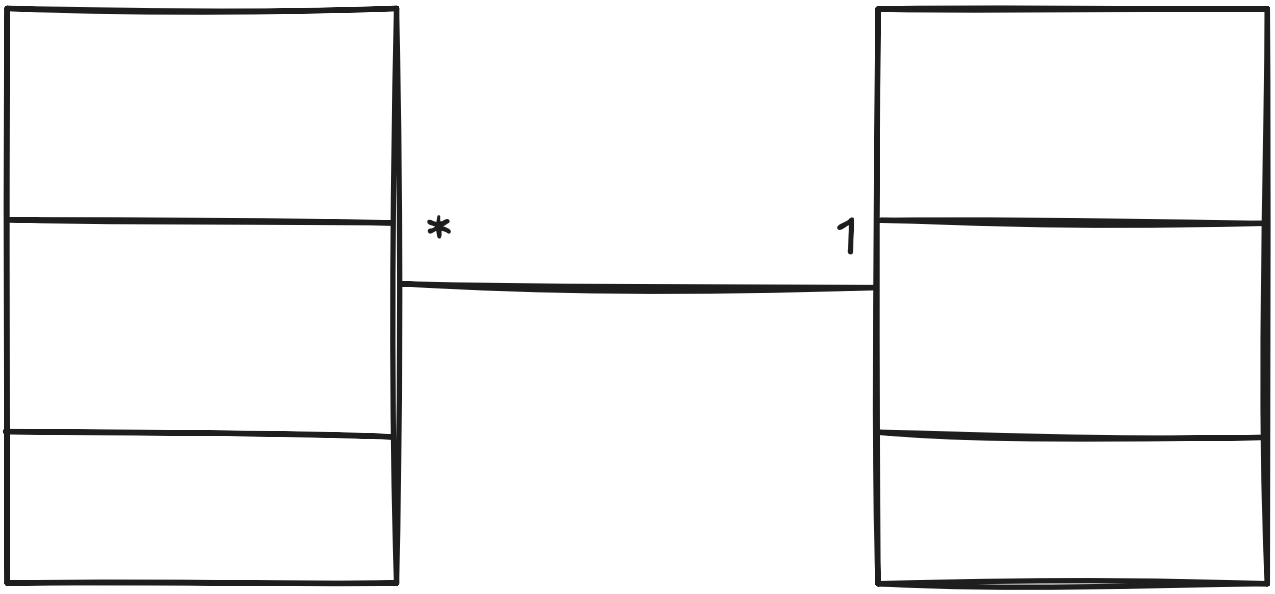
Ассоциация - *семантическое* отношение между классами. Самое *мягкое* отношение. Этому отношению *характерна мощность*, то есть соответствие одного класса другому классу. Самая мощная сцепка - это *один к одному*. *Отношение мощность характеризует* понятие *деструктуризации* объекта. То есть это показывает насколько всё сломается если убить один из объектов.



Существуют следующие *типы отношений*:

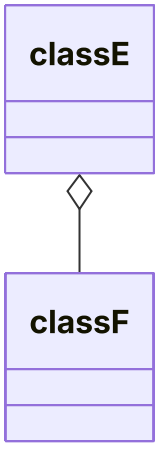
- `1 - 1` - *один к одному*.
- `1 - *` - *один к многим*. Самый лучший тип отношений.
- `* - 1` - *многие к одному*.
- `* - *` - *многие ко многим*. Самый худший тип отношений, потому что мы не можем предсказать поведение сущностей при удалении одной из них.

Если что *мощность отображается как цифры* (или *) на концах связи, но mermaid не позволяет мне их нарисовать. Примерно так:



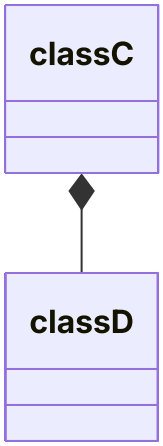
Агрегация

Агрегация - *непосредственное отношение* между объектами и подразумевает *включение по ссылке*. При этом включение может быть не обязательным. Проблема с `Null`, что бы её решить используется `Nothing Obj`.



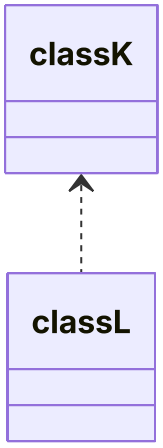
Композиция

Композиция - непосредственное *обязательное включение*. Например, если мы *кодим таблицу*, то *заголовок* будет включаться с помощью *композиции*, а *строки* с помощью *агрегации*.



Зависимость

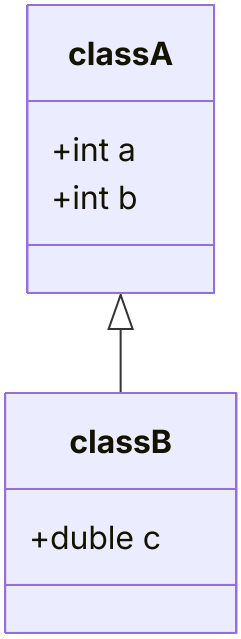
Зависимость - явная зависимость. Предполагает *использование* экземпляров *класса от которого зависит в своих функциях*.



Наследование

Наследование - непосредственное наследование от родительского класса. *Отношение* между классами, которое *позволяет создавать* классы *наследников* на основе родительских. Путём наследования либо структуры, либо поведения, либо всего вместе. На практике наследование существует ради двух целей:

- *Результат рефакторинга*.
- *Выделение интерфейсов*.
  Самый жёсткий вид отношений, так как он формируется на этапе компиляции.



ⓘ Примечание

При наследовании размеры типов внутренних, включая наследников, складываются. `sizeof(CalssA) = 8`, `sizeof(CalssA) = 16`.